

Claude Code Agents – Pedagogical Overview

How parallel AI software work is organized

1. The building blocks



Subagents

Specialized workers inside one Claude Code session



Agent View

Dashboard for many background Claude Code sessions



Agent Teams

A lead agent coordinates multiple collaborating sessions



Dynamic Workflows

JavaScript orchestration scripts that spawn and coordinate many agents



Worktrees

Git-isolated working directories for safe parallel edits

2. When to use what



Small side task



Subagent



Several independent tasks in parallel



Agent View



Multi-agent collaboration



Agent Team



Large repeatable orchestration



Dynamic Workflow



Parallel coding without collisions



Worktrees

3. What a custom subagent can define



Role / Instructions



Model



Tools



Permissions



Custom Subagent



Hooks



Memory



Isolation



MCP / Skills

A subagent is more than a prompt: it is a packaged execution unit.

4. Safety & governance



Least-privilege tool access



Hooks can block unsafe commands



Background agents auto-deny actions needing permission



Worktrees prevent file-edit collisions



Project memory can capture institutional knowledge

Policy moves from prose into runtime controls.



Subagents

= workers



Agent View

= control tower



Agent Teams

= collaborative squad



Workflows

= automation pipeline



Worktrees

= isolated workspaces

5. The big takeaway for agentic engineering

From



• AI pair programmer



To



• Agentic software delivery system

New engineering disciplines



Role design



Tool governance



Context isolation



File isolation



Quality gates



Orchestration



Cost control

The bottleneck shifts from coding alone to orchestration, review, permissions, context, and governance.



Subagents = workers



Agent View = control tower



Agent Teams = collaborative squad



Workflows = automation pipeline



Worktrees = isolated workspaces

The Scaling Problem: Escaping the Single Context Window

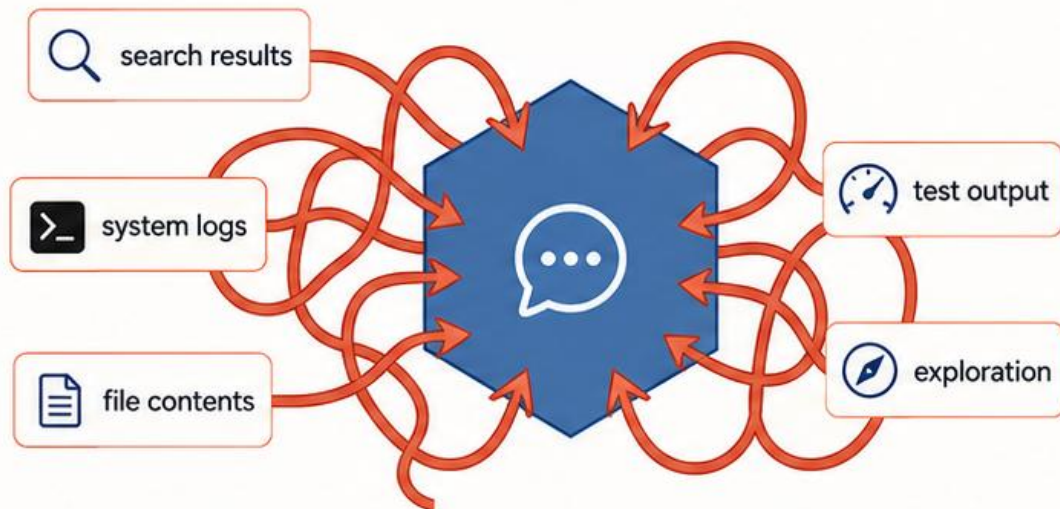
Why agentic software work moves from one overloaded chat to coordinated parallel sessions



Parallel Claude sessions increase throughput — but also multiply token usage, coordination, and governance needs.

1. Linear chat

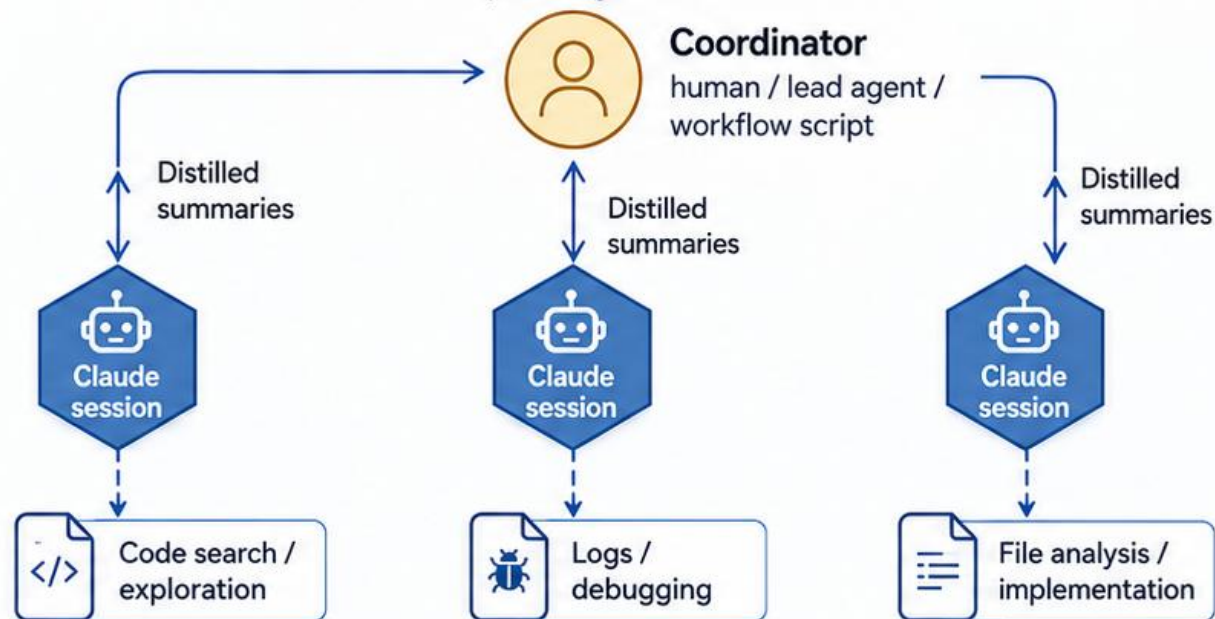
Everything floods one context



Context flood

2. Parallel agent work

The paradigm shift



Single chat =
one overloaded context



Subagents =
isolated workers



Summaries =
compressed signal back



Worktrees =
safe parallel coding

The Orchestration Matrix



Same worker substrate, different orchestration topology: who coordinates, who communicates, and how work is verified.

Topology	Coordinator	Inter-Agent Comms	Best For
 Subagents	 Main Claude session	 None	 Side tasks that would flood main context
 Agent View (Research Preview)	 Human	 None	 Independent tasks you monitor later
 Agent Teams (Experimental)	 Lead agent	 Direct messaging	 Complex projects requiring worker coordination
 Dynamic Workflows (Research Preview)	 Script	 Script-mediated verification	 Large audits, migrations, cross-checked research



Subagents = delegated specialists



Agent View = human-monitored sessions



Agent Teams = collaborative coordination

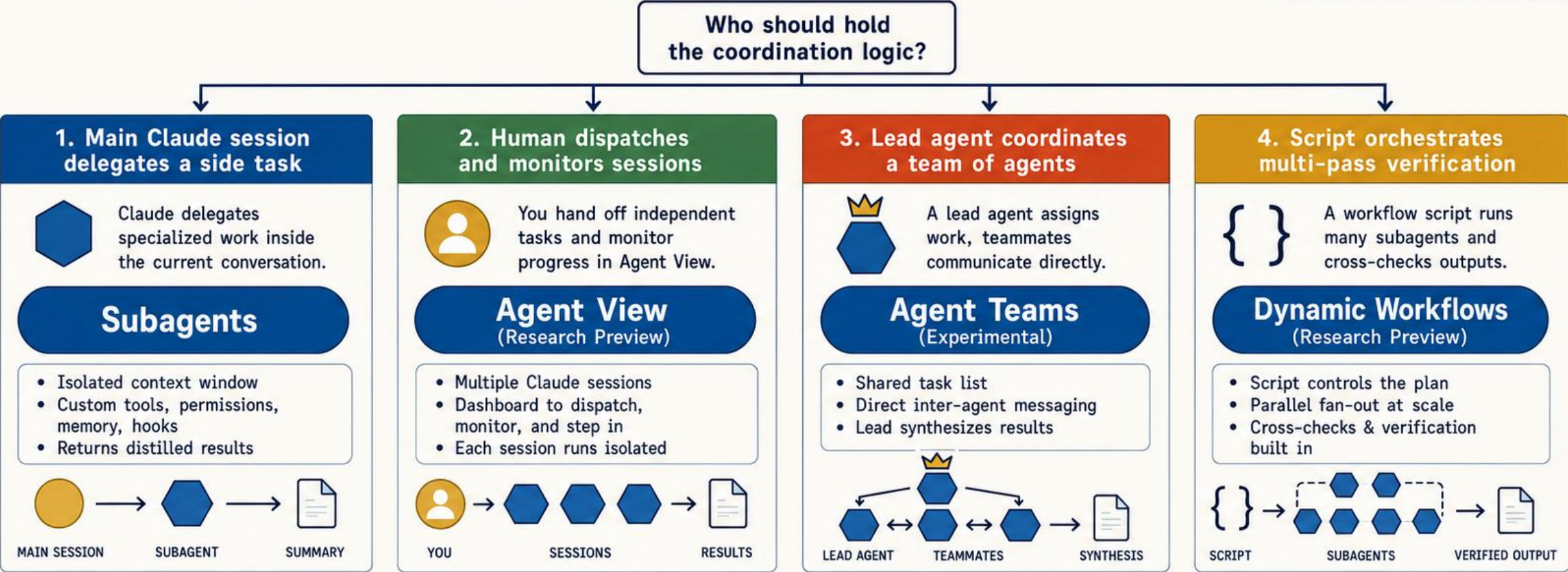


Dynamic Workflows = scripted orchestration

Architectural Decision Tree: Choose Your Orchestration Topology

Different coordination, different communication, different guarantees.

Methodology from
agentic-SDLC.it

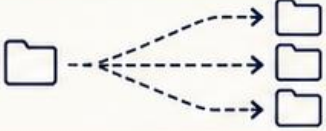




Will tasks edit the same files in parallel?

YES → Use **Git Worktree Isolation**

NO → Standard working directory is fine



Worktrees prevent parallel edits from colliding.



Safe scaling requires boundaries.
Use the right tools, permissions, hooks, skills, and memory for every agent.



The right topology accelerates delivery.
The right controls ensure quality.

Learn more and adopt the full Agentic SDLC methodology at **agentic-SDLC.it**

Topology 1: Subagents

Delegated workers inside a single Claude Code session.

Methodology from
agentic-SDLC.it



Runs in its own isolated context.

The subagent has a separate context window and tools to do deep exploration.



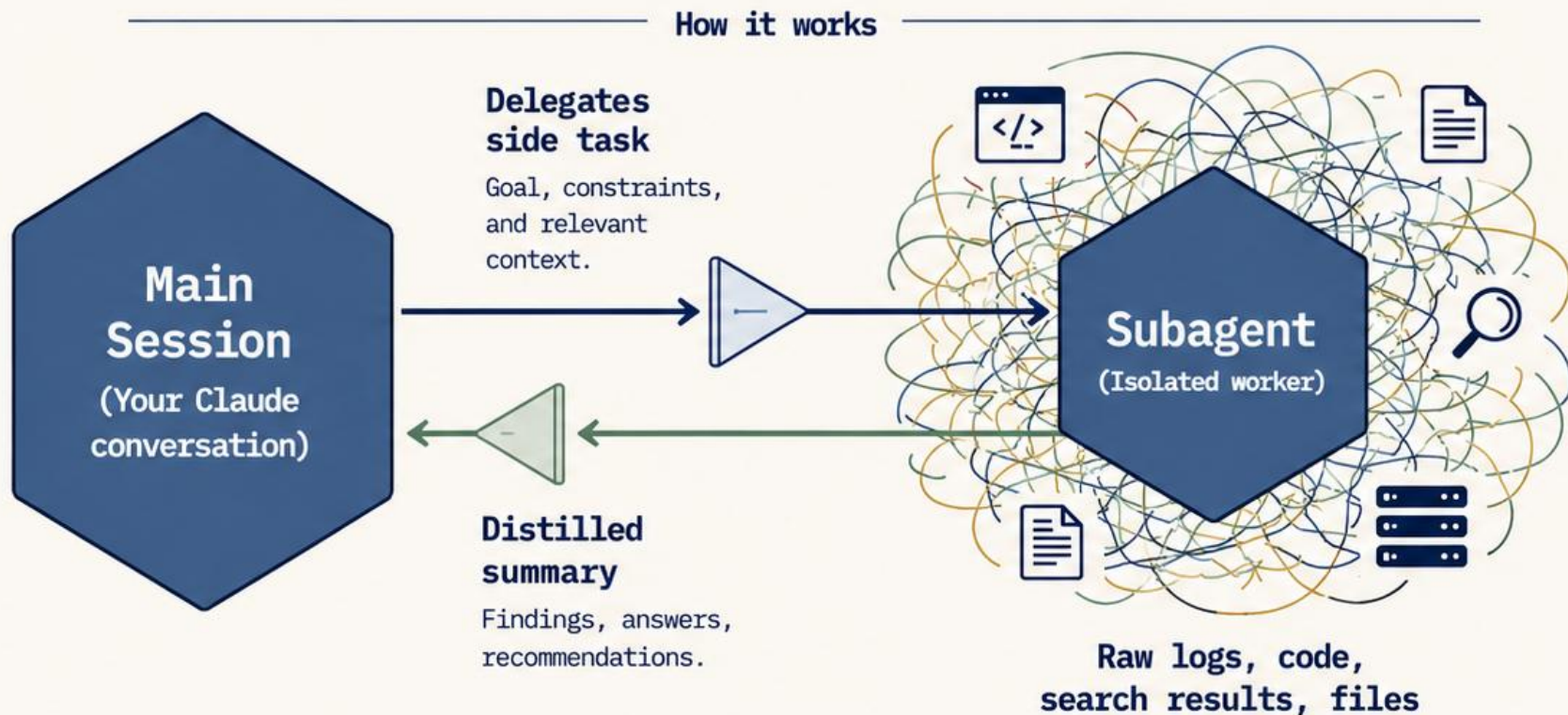
Protects your main conversation.

No noisy logs, search results, or file dumps flood the main context.



Returns only a distilled summary.

You get the signal, not the noise.



Tools

(Read, Grep, Bash, etc.)



Permissions

(Restricted or inherited)



Subagents can have their own

Model

(haiku, sonnet, opus, etc.)



Memory

(user, project, or local)



Hooks

(Pre/Post tool controls)



Skills

(Reusable capabilities)



Use subagents whenever a task would flood your main context with low-signal details.
You stay in flow. The subagent handles the noise.

Learn more about the full methodology at
agentic-SDLC.it

Topology 2: Agent View [Research Preview]

Human-supervised background sessions.

Methodology from
agentic-SDLC.it

1



Unified control plane.

Dispatch and monitor multiple background sessions from one unified screen.

2



Independent by default.

Workers (sessions) process independently and do not communicate with each other. They report only to you.

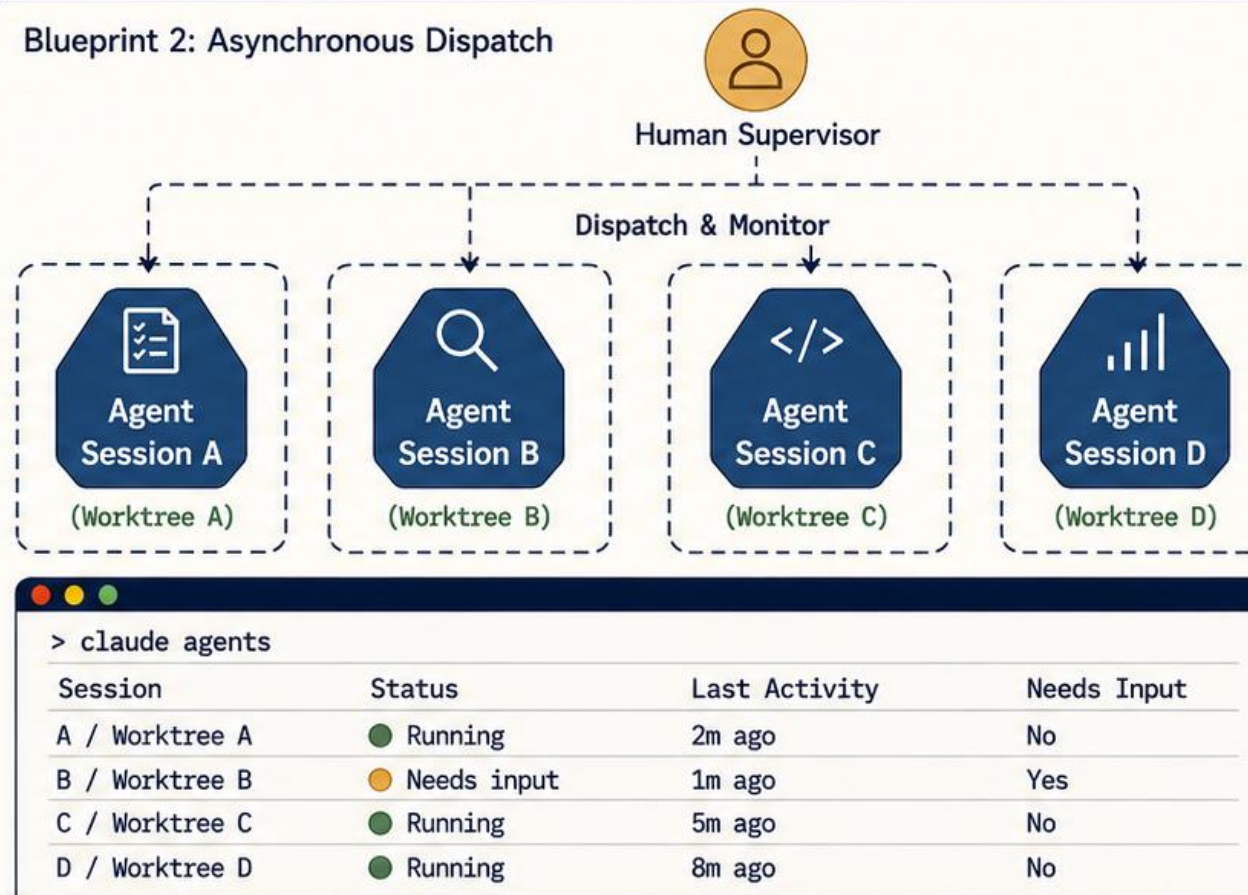
3



Supervise, step in, continue.

Best for handing off independent tasks and stepping in only when a worker explicitly needs input. Detach and let it run again.

Blueprint 2: Asynchronous Dispatch



How it flows



1. You dispatch independent tasks.



2. You monitor progress and status.



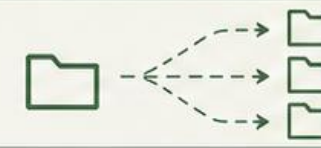
3. You step in only when a session needs input.



4. Detach and let the session continue.



Isolation by Worktree. Each dispatched session runs in its own Git worktree, preventing parallel file-edit collisions and keeping changes isolated.



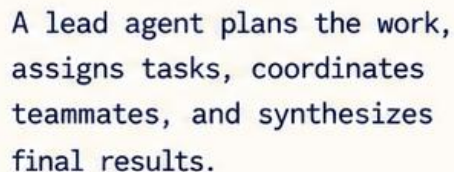
Agent View = human-controlled parallelism. You stay in control, keep context clean, and scale throughput with confidence.

Learn more about the full methodology at

agentic-SDLC.it

Methodology from
agentic-SDLC.it

1 Lead-agent coordination.



Teammates have independent context windows and can message each other directly to sync on dependencies and findings.



Use when parallel workers must share findings, challenge each other, or coordinate across layers or modules.



Agent Teams do not isolate teammates in worktrees.
Partition file ownership so teammates do not edit the same files.



Higher coordination overhead and token usage.
Not ideal for tightly coupled or sequential work.



Share context, challenge assumptions, and ship better outcomes—together.

The diagram illustrates a multi-agent system architecture. At the top, a **Human Supervisor** (represented by a person icon) interacts with a **Lead Agent** (represented by a blue hexagon). The Lead Agent is connected to a box listing its functions:

- Sets direction & priorities
- Breaks work into tasks
- Coordinates execution
- Synthesizes results

The Lead Agent is connected to three **Teammates** (represented by blue hexagons labeled 1, 2, and 3). Teammate 2 is connected to a **Shared Task List** (represented by a clipboard icon). The Shared Task List is connected to Teammate 3. Dashed arrows indicate **Direct Inter-Agent Messaging** between Teammate 1 and Teammate 2, and between Teammate 2 and Teammate 3. A dashed arrow also points from Teammate 3 back to Teammate 1.

Learn more about the full methodology at

agentic-SDLC.it

Topology 4: Dynamic Workflows [Research Preview]

Scripted, multi-pass cross-checking at scale.

Methodology from
agentic-SDLC.it

1



A script drives the plan.

The execution plan lives in code, not in Claude's turn-by-turn judgment.

2



Intermediate outputs feed verifier agents.

Early-stage results are explicitly passed to verifier, reviewer, or synthesizer agents.

3



Built for massive, multi-pass work.

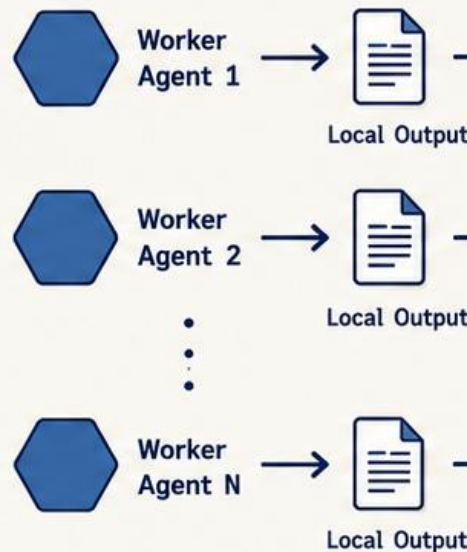
Ideal for codebase-wide audits, 500-file migrations, or research with independent cross-checks.

Blueprint 4: Scripted Orchestration



A workflow script orchestrates many subagents and data flows.
e.g., 500-file migration or codebase-wide audit

Phase 1: Parallel Audit



Phase 2: Verification & Consolidation



Phase 3: Final Output



Final Report /
Change Plan

Deliverables

- Actionable report
- Changes / patches
- Migration plan
- Evidence & logs

Feedback loop for additional passes (if needed)



Orchestration as code.

Workflows are inspectable, approvable, and reusable. You can version, share, and evolve them over time.



Use when it matters.

Great for high-value work that needs multiple passes or independent verification.

Learn more about the full methodology at

agentic-SDLC.it

The Isolation Imperative: Git Worktrees

Parallel Claude sessions need separate working copies to avoid file-edit collisions.

Learn the full methodology at
agentic-SDLC.it



Architectural Flaw: Shared Working Copy

Multiple sessions edit the same files in one checkout.



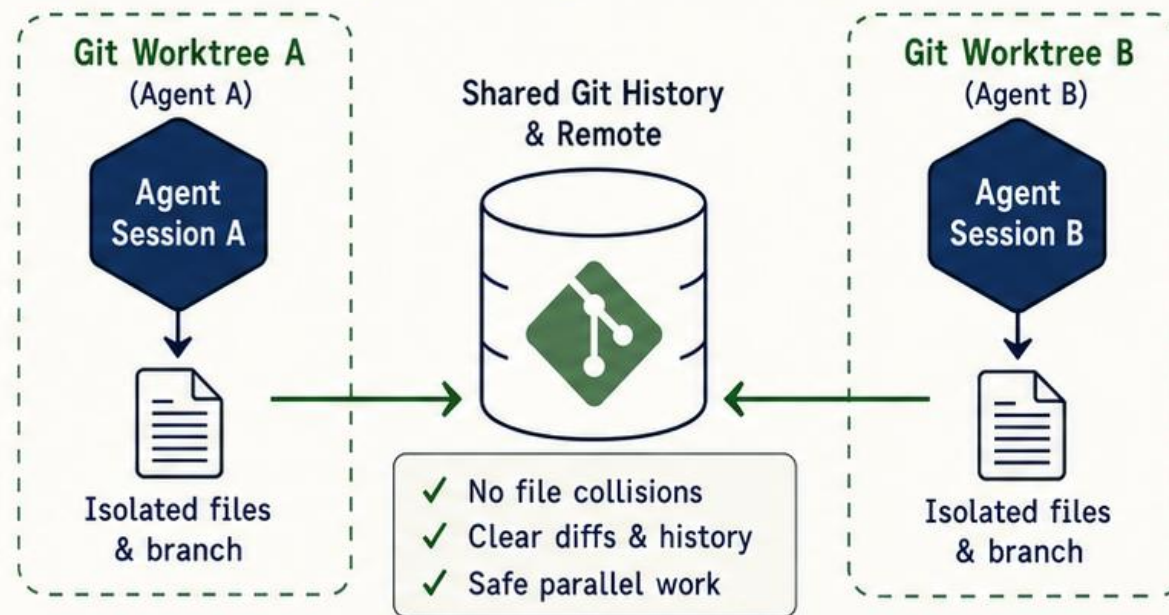
Same files, same checkout

- Conflicting edits
- Overwritten changes
- Hard to reason about diffs



Architectural Safeguard: Isolated Worktrees

Each session gets its own git worktree (separate checkout).



Agent View

Automatically moves each dispatched session into its own worktree.



How Worktrees Are Used

Subagents

Can be configured to run in their own worktree for extra isolation.



Agent Teams (Exception)

Teammates are not auto-isolated. Partition file ownership instead.



Isolation by design.

Git worktrees let Claude Code scale safely across agents and sessions while keeping your codebase consistent and your history clean.

Learn more about the full methodology at

agentic-SDLC.it

Defining the Boundaries

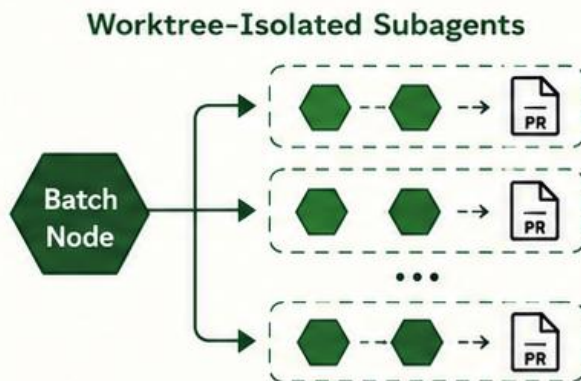
Supporting mechanisms that enable parallel work—without being orchestration topologies.

Learn more at
agentic-SDLC.it

1 /batch



A packaged skill that uses 5–30 worktree-isolated subagents to split large changes and open PRs. Not a coordination style of its own.

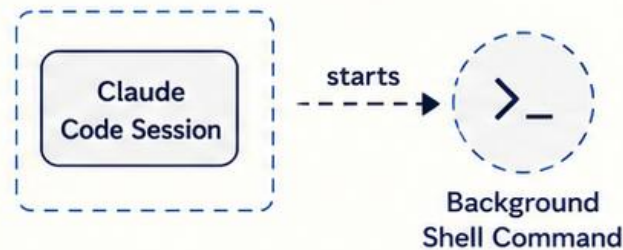


Uses subagents + worktrees under the hood.
Not an independent topology.

2 Background bash



Runs one shell command in the background. It doesn't block your conversation and does not spawn an agent.

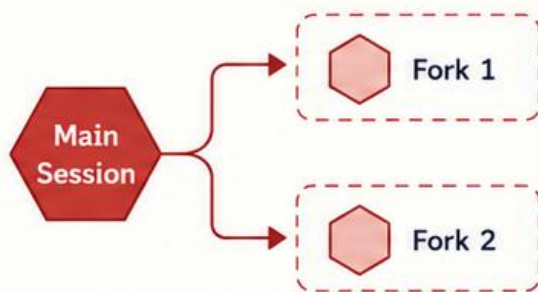


Great for long-running commands.
Not an agent. Not a topology.

3 Forked subagent



A subagent fork that inherits your full conversation context instead of starting fresh. It's a spawn mode, not a distinct surface.

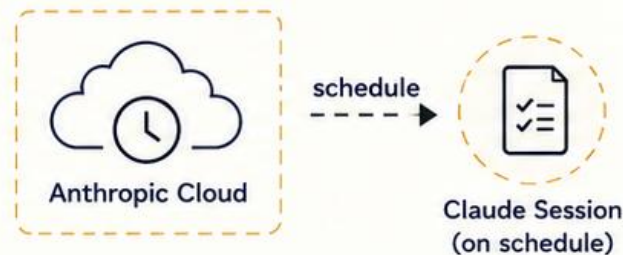


Shares full context and goals.
Not a separate coordination topology.

4 Routine



Runs a Claude session on a schedule in Anthropic's cloud infrastructure. Solves automation, not local parallel execution.



Cloud-scheduled automation.
Not local parallel orchestration.



Boundary Rule: It's about who coordinates.

Topologies decide who orchestrates work (Claude, human, lead agent, or script). These tools support that work—but are not new topologies.

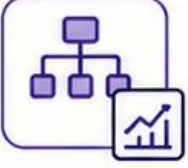
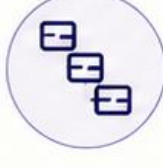
Explore the full methodology and decision guide at

agentic-SDLC.it

The Command Center

Visibility depends on your execution scope. Use the right telemetry command for your orchestration topology.

Learn more at
agentic-SDLC.it

CLI / Slash Command	What It Shows	Execution Scope	Best For
 > claude agents (CLI) Opens Agent View	Background Sessions Every session, state, and input need across your Agent View dashboard. See what's running beyond this session.	 Across all background Claude sessions	✓ Monitor Agent View sessions ✓ Inspect state & inputs ✓ Manage & resume work
 /agents (Slash) Opens Subagents & Library	Current-Session Subagents Live subagents in this session and your reusable subagent library. See who you spawned (and reuse).	 Inside the current Claude session	✓ Check active subagents ✓ View conversation & status ✓ Create or edit subagents (Library tab)
 /tasks (Slash) Open Background Items	Current-Session Background Items All background tasks started in this session: check, attach to, or stop. Keep your long tasks in view.	 Inside the current Claude session	✓ Manage running tasks ✓ Attach to a task ✓ Stop or clean up work
 /workflows (Slash) Open Workflow Runs	Workflow Runs Dynamic workflow runs, phase status, and completed agent count. Track complex, multi-pass work.	 Across workflow runs (current or completed)	✓ Monitor phase progress ✓ See completed agents ✓ Review run history
 Pick the right lens. Different scopes, different tools. Use the command that matches how your work is orchestrated.	 Agent View → <code>claude agents</code>	 Subagents → <code>/agents</code>	 Background Items → <code>/tasks</code>
			 Workflows → <code>/workflows</code>
 Right command. Right context. Better visibility. Know what's running, where it's running, and what to do next.	Explore the full methodology, decision guide, and examples at agentic-SDLC.it		